MCP Server Suite for the MahaSamvaad GR Chat API

Technical Specification — v1.0 Internal QA/Observability, Grounding-Score, and Reformulation-Robustness tooling built around `POST /api/v1/chat`

1. Purpose & Scope

This document specifies three Model Context Protocol (MCP) servers built around the staging MahaSamvaad chat endpoint. Together they give the team an internal, agent-accessible layer for logging every call, scoring answer quality, and probing retrieval stability — the observability and evaluation surface that the production chat API does not provide on its own, since the API itself is stateless across calls.

In scope	Server 1 (QA/Eval/Observability), Server 2 (Grounding/Citation Coverage), Server 3 (Query Reformulation Tester); the shared SQLite store; MCP tool and resource contracts; deployment on the staging network.
Out of scope	Changes to the MahaSamvaad agent, the ingestion pipeline, or the production `/chat` API. The SRS's own Phase 1–3 evaluation harness (LLM-as-judge, RLHF loop) is a separate, larger effort — referenced only where it overlaps.

---

2. Background & System Context

MahaSamvaad is a multi-pipeline conversational agent answering questions over Maharashtra Government Resolutions (GRs), Acts, and Schemes, with optional Serper web-search fusion. Per the project SRS, the agent already performs intent classification, hybrid corpus retrieval (Qdrant + FalkorDB), reranking, and RAG+web fusion, and is instrumented with Langfuse tracing at the node level. What it does **not** expose today is a queryable history of past interactions, a hallucination-risk signal per answer, or a way to test whether rephrasing a question changes which document gets cited — those three gaps are exactly what this spec addresses.

2.1 Endpoint under test

Item	Value
Base URL (staging)	`http://20.40.56.211:8000`

Item	Value
Swagger / OpenAPI	`http://20.40.56.211:8000/docs`
Primary endpoint	`POST /api/v1/chat`
Streaming variant	`POST /api/v1/chat/stream`
Health check	`GET /health`
Query parameter	`web_search: boolean` (default `true`) — Serper fusion on/off; `false` = corpus-only

2.2 Request / response contract (as observed via Swagger)

This is the contract every server in this spec builds on. Fields whose exact backend semantics could not be confirmed from Swagger alone are called out in the risk notes under each tool below rather than assumed.

```
ChatRequest {
  user_id: string
  chatroom_id: string
  user_message_id: string
  message_id: string
  message: string
  model_name: string
  history: HistoryMsg[]           // { role, content, sources[] }
  last_turn_filepaths: string[]
}

ChatResponse {
  response: string
  last_turn_filepaths: string[]
  metadata: ChatMetadata {
    intent, language, route_hint
    sources: CitationOut[]        // { citation_id, filename, filepath,
                                  //   page_number, gr_number, date,
                                  //   department, anchors: AnchorOut[] }
    web_sources: WebSourceOut[]   // { ref, index, title, link, snippet, date }
    latency_seconds, model, tokens: TokenUsage
    message_id, langfuse_trace_id, lineages, lineage_data
  }
}

AnchorOut { page_number: int, start_text: string, end_text: string }
```

2.3 Why these three servers, in this order

1. **Server 1 is the foundation.** It is the only component that persists history; the API itself is stateless across calls. Servers 2 and 3 both read data that Server 1 writes, so it must exist first.

2. **Server 2 turns an already-returned field into a metric.** `anchors[].start_text/end_text` becomes an actionable hallucination-risk score — no new model call required for v1.
3. **Server 3 is a retrieval-robustness probe, independent of Server 2.** It tests whether the *retrieval layer* is stable under paraphrase, not whether the *generated text* is well-grounded — a distinct failure mode from Server 2's.

---

3. Design Principles

- **MCP-native, not script-native.** Every capability is exposed as a tool or resource so any MCP-compatible client — a dashboard UI, a debugging agent, or a CI job — can call it mid-conversation, not just a bespoke script.
- **One shared store, three servers.** All three servers read/write the same SQLite database (`store.db`) rather than three siloed databases, so grounding scores and reformulation runs can be joined back to the original logged query by `message_id`.
- **Server 1 is the only conceptual owner of `query_log`.** Servers 2 and 3 reuse the same logging code path (see §8) rather than re-implementing it, so every row — regardless of which server triggered the `/chat` call — lands in one consistent table.
- **Fail open, log everything.** A malformed response, a timeout, or an empty `sources[]` array must never crash a tool call. It is logged as a flagged/error row and returned to the caller as a structured result, never a raised exception across the MCP boundary.
- **v1 is deliberately simple, with named upgrade paths.** `difflib` string overlap instead of embeddings, SQLite instead of Postgres, 3–4 paraphrases instead of a generation pipeline — each is a conscious v1 choice with an explicit v2, not an oversight.

---

4. Shared Infrastructure

All three servers are separate MCP processes but share two things on disk: the `store.db` SQLite file and a common chat-client module. Get this contract right first and each server becomes a thin layer on top of it.

4.1 Data store — `store.db` (SQLite, WAL mode)

SQLite is sufficient for a staging/eval workload (single-writer-friendly with WAL, zero ops, trivially copyable into a report appendix). Swap for Postgres only if concurrent write load from multiple agents becomes real.

`query_log` — one row per `/chat` call, written by `query_and_log`:

```
CREATE TABLE query_log (
  id                INTEGER PRIMARY KEY AUTOINCREMENT,
  called_at         TEXT NOT NULL,            -- ISO-8601 UTC
  user_id           TEXT,
  chatroom_id       TEXT,
  user_message_id   TEXT,
  message_id        TEXT,                     -- from response.metadata.message_id
  message            TEXT NOT NULL,
  model_name        TEXT,
  web_search        INTEGER,                  -- 0/1, the query param used
  response_text     TEXT,
  intent            TEXT,
  language          TEXT,
  route_hint        TEXT,
  department        TEXT,                     -- derived: top source's department
  sources_json      TEXT,                     -- raw CitationOut[] as JSON
  web_sources_json  TEXT,                     -- raw WebSourceOut[] as JSON
  prompt_tokens     INTEGER,
  completion_tokens INTEGER,
  total_tokens      INTEGER,
  latency_seconds   REAL,
  langfuse_trace_id TEXT,
  http_status       INTEGER,                  -- 200, 4xx, 5xx, or -1 for network error
  error_text        TEXT,                     -- non-null only on failure
  raw_response_json TEXT                      -- full ChatResponse, for replay/debug
);
CREATE INDEX idx_query_log_time   ON query_log(called_at);
CREATE INDEX idx_query_log_intent ON query_log(intent);
CREATE INDEX idx_query_log_dept   ON query_log(department);
CREATE INDEX idx_query_log_msgid  ON query_log(message_id);
```

`grounding_scores` — one row per `score_grounding` call (Server 2):

```
CREATE TABLE grounding_scores (
  id               INTEGER PRIMARY KEY AUTOINCREMENT,
  message_id       TEXT NOT NULL REFERENCES query_log(message_id),
  scored_at        TEXT NOT NULL,
  coverage_pct     REAL,                       -- 0-100, NULL if no sources to ground
against
  total_sentences  INTEGER,
  grounded_count   INTEGER,
  ungrounded_json  TEXT,                       -- [{sentence, best_match_score}]
  method           TEXT DEFAULT 'difflib_v1'     -- upgrade path: 'embedding_v2'
);
CREATE INDEX idx_ground_msgid ON grounding_scores(message_id);
```

`reformulation_runs` / **`reformulation_variants`** — Server 3:

```
CREATE TABLE reformulation_runs (
  id              INTEGER PRIMARY KEY AUTOINCREMENT,
  run_at          TEXT NOT NULL,
  original_query  TEXT NOT NULL,
  stability_score REAL,                          -- fraction agreeing with majority GR
  majority_gr     TEXT,
  odd_one_out_ids TEXT                            -- JSON list of variant ids that
disagreed
);

CREATE TABLE reformulation_variants (
  id            INTEGER PRIMARY KEY AUTOINCREMENT,
  run_id        INTEGER NOT NULL REFERENCES reformulation_runs(id),
  variant_text  TEXT NOT NULL,
  variant_kind  TEXT,                             -- formal | informal | marathi | reorder
  message_id    TEXT,                             -- FK into query_log for this call
  top_gr_number TEXT,
  top_filepath  TEXT
);
```

4.2 Common chat client — `mcp_common/chat_client.py`

A single module wraps `POST /api/v1/chat` and is imported by all three servers, so retry/timeout/error-shaping logic exists in exactly one place.

```
async def call_chat(
    message: str,
    *,
    web_search: bool = True,
    model_name: str | None = None,
    history: list[dict] | None = None,
    last_turn_filepaths: list[str] | None = None,
    chatroom_id: str | None = None,
    user_id: str = "mcp-eval-bot",
) -> ChatResult:
    ...
```

- Generates `user_message_id` / `message_id` (uuid4) client-side when the caller does not supply one, so every logged row is joinable even on the first turn.
- **Timeout:** 60s connect+read (corpus + web-fusion calls can be slow). **Retry:** one retry on connection error or 5xx with 2s backoff; no retry on 4xx (a bad request will not succeed on retry).
- **Never raises across the MCP tool boundary.** On failure it returns `ChatResult(ok=False, error=..., http_status=...)` so `query_and_log` can still write a row — a failed call is itself a signal worth graphing (§5.2).

4.3 Configuration (environment variables, shared across all three servers)

Variable	Purpose	Default
`MAHASAMVAAD_BASE_URL`	Staging endpoint base	`http://20.40.56.211:8000`
`STORE_DB_PATH`	Shared SQLite file	`./data/store.db`
`MCP_DEFAULT_USER_ID`	`user_id` sent when caller omits one	`mcp-eval-bot`
`MCP_HTTP_TIMEOUT_S`	Per-call timeout	`60`
`GROUNDING_METHOD`	`difflib_v1` | `embedding_v2`	`difflib_v1`
`GROUNDING_THRESHOLD`	Min match ratio to count a sentence as grounded	`0.55`
`LOG_LEVEL`	Python logging level for all three servers	`INFO`

4.4 Directory layout

```
mahasamvaad-mcp/
├── mcp_common/
│   ├── chat_client.py       # §4.2 — shared HTTP wrapper
│   ├── store.py             # SQLite connection + schema migration
│   └── config.py            # env var loading (§4.3)
├── server_observability/    # Server 1 — §5
│   └── main.py
├── server_grounding/        # Server 2 — §6
│   └── main.py
├── server_reformulation/    # Server 3 — §7
│   └── main.py
└── data/
    └── store.db
```

---

5. MCP Server 1 — Internal QA / Eval & Observability

`mcp-server: mahasamvaad-observability` · transport: stdio (dev) / streamable-http (staging) · depends on: nothing (foundation layer)

Every other tool in this spec depends on this server existing first — it is the only source of historical data, since the `/chat` API itself is stateless across calls.

5.1 Tool — `query_and_log`

Thin wrapper around `POST /api/v1/chat`. Every call is written to `query_log` with the full response, regardless of success or failure.

Input schema

Field	Type	Notes
`message`	string, required	the user question to send
`web_search`	boolean, default `true`	passed through as the query param
`model_name`	string, optional	forwarded if the caller wants a specific model
`history`	array, optional	prior turns; passed through untouched
`last_turn_filepaths`	array[string], optional	for multi-turn continuity tests
`chatroom_id` / `user_id`	string, optional	defaults generated if omitted (§4.2)

Output

```
{
  "ok": true,
  "message_id": "…",
  "response": "…full answer text…",
  "intent": "…", "language": "…", "route_hint": "…",
  "sources": [ { "gr_number": "…", "filename": "…", "department": "…", "…": "…" } ],
  "web_sources": [ "…" ],
  "tokens": { "prompt_tokens": 0, "completion_tokens": 0, "total_tokens": 0 },
  "latency_seconds": 0.0,
  "langfuse_trace_id": "…",
  "logged_row_id": 123
}
```

On failure (timeout, non-200, malformed JSON): `ok:false`, `error`, `http_status` are returned, and the row is still written with `error_text` populated.

5.2 Tool — `get_dashboard_stats`

Reads `query_log` (optionally joined with `grounding_scores` and `reformulation_runs`) and returns aggregates. Pure read — never calls the live API.

Parameter	Type	Default	Effect
`since`	ISO date, optional	last 7 days	lower bound on `called_at`

Parameter	Type	Default	Effect
`group_by`	enum: `intent` | `department` | `day`	`intent`	grouping for the volume/latency breakdown
`outlier_z`	float, optional	`2.0`	z-score threshold for the outlier list

Returned aggregates

- **Latency percentiles** — p50 / p90 / p99 of `latency_seconds` over the window.
- **Token cost over time** — daily sum of `total_tokens` (and prompt/completion split) to spot cost spikes.
- **Query volume** by `intent` / `department` / `day` — count grouped per `group_by`.
- **Error rate** — % of rows with `http_status != 200` or non-null `error_text`, in the window.
- **Outlier queries** — rows where `latency_seconds` or response length deviate more than `outlier_z` standard deviations from the group mean, returned with `message_id`, `message`, `latency_seconds`, and a reason tag (`high_latency` | `short_response` | `long_response`).

Outlier pass is computed in Python (SQLite has no native `STDDEV`):

```
rows = db.execute(
    "SELECT id, message_id, message, latency_seconds, length(response_text) AS rlen "
    "FROM query_log WHERE called_at >= ?", (since,)
).fetchall()
mean, std = statistics.mean(latencies), statistics.pstdev(latencies)
outliers = [r for r in rows if abs(r.latency_seconds - mean) > outlier_z * std]
```

5.3 Resource — `query_log://recent`

Exposed as a browsable MCP resource (not a tool call), so a client can pull up recent history with zero setup.

URI	Behavior
`query_log://recent`	last 50 rows, newest first, compact projection (`message_id`, `message`, `intent`, `department`, `latency_seconds`, `http_status`)
`query_log://recent?limit=200`	override the row count (max 500, to keep the resource payload bounded)

URI	Behavior
`query_log://recent?intent=policy_exploration`	filter by intent before truncating to `limit`

Resources are read-only and side-effect-free by MCP convention — this one never triggers a live `/chat` call, it only reads `store.db`.

5.4 Why this matters as MCP infrastructure, not a script

Because `query_and_log` and `get_dashboard_stats` are MCP tools rather than a bespoke CLI, any MCP-compatible agent can call them mid-conversation — e.g. a debugging agent can call `get_dashboard_stats` before answering "is the system currently degraded?", or the reformulation server can call `query_and_log` directly instead of re-implementing the HTTP call. This is the reason the suite is three MCP servers instead of three unrelated Python scripts.

---

6. MCP Server 2 — Grounding / Citation Coverage Score

`mcp-server: mahasamvaad-grounding` · transport: stdio / streamable-http · depends on: `store.db` written by Server 1

A hallucination-risk signal per response, computed entirely from data the endpoint already returns — no extra model call needed for v1. In a government-facing tool, an ungrounded sentence in an answer about GR requirements is a compliance liability, not just a quality nitpick, which is why this ships before anything fancier.

6.1 Tool — `score_grounding`

Field	Type	Notes
`message_id`	string, required	must exist in `query_log` (written by `query_and_log`)
`method`	enum, optional	`difflib_v1` (default) | `embedding_v2` — overrides `GROUNDING_METHOD`

Algorithm (v1 — `difflib_v1`)

1. Look up `response_text` and `sources_json` for `message_id` in `query_log`; parse `sources[].anchors[]` into a flat list of `(start_text, end_text)` spans.
2. Split `response_text` into sentences. Use a splitter that handles both the Devanagari danda (`।`) and Latin full stops — do not assume English-only punctuation, since MahaSamvaad answers can be

bilingual.

3. For each sentence, compute `difflib.SequenceMatcher(None, sentence, anchor_text).ratio()` against every anchor's `start_text + end_text` span; keep the best match.
4. A sentence is grounded if `best_match_score ≥ GROUNDING_THRESHOLD` (default `0.55` — tune against a hand-labelled sample before trusting the default, see §12).
5. `coverage_pct = grounded_count / total_sentences × 100`. Write one row to `grounding_scores`; return the flagged ungrounded sentences with their best score so a reviewer can distinguish near-misses from clean fabrications.
6. If `sources[]` is empty (e.g. an off-topic/general-intent response with nothing to ground against), short-circuit to `coverage_pct: null` with `reason: "no_sources_to_ground_against"` — never divide by zero.

Output

```
{
  "message_id": "…",
  "coverage_pct": 71.4,
  "total_sentences": 7,
  "grounded_count": 5,
  "ungrounded_sentences": [
    { "sentence": "…", "best_match_score": 0.31 },
    { "sentence": "…", "best_match_score": 0.44 }
  ],
  "method": "difflib_v1"
}
```

v2 upgrade path (`embedding_v2`): replace step 3 with cosine similarity between sentence and anchor embeddings (a small multilingual sentence-transformer) to catch paraphrased-but-grounded sentences that string overlap misses. Same output shape — only `method` and how `best_match_score` is computed change.

6.2 Tool — `grounding_trend`

Aggregates `score_grounding` results stored over time, sliced by department or query type — turning a one-off score into a trend the team can act on. "Grounding drops to 40% for Urban Development Dept queries" is an actionable finding; a single score is not.

Parameter	Type	Default	Effect
`slice_by`	enum: `department` | `intent` | `route_hint`	`department`	grouping column, joined from `query_log`

Parameter	Type	Default	Effect
`since`	ISO date, optional	last 30 days	lower bound on `scored_at`
`min_samples`	int, optional	`5`	groups with fewer scored responses are omitted (avoids noisy single-sample groups)

Output: per-slice mean/median `coverage_pct`, sample count, and trend direction (7-day rolling average vs. prior period), plus the single worst-scoring `message_id` per slice as a concrete example for a report.

---

7. MCP Server 3 — Query Reformulation Tester

`mcp-server: mahasamvaad-reformulation` · transport: stdio / streamable-http · depends on: `query_and_log` (Server 1) for each fired variant

A retrieval-robustness probe, independent of the LLM's phrasing of the answer: it tests whether the retrieval layer returns the same government document regardless of how the question is worded. This is the most demoable of the three servers — a concrete, reproducible case where rephrasing a question changed which GR got cited is a far stronger report finding than a general "routing seems inconsistent" claim.

7.1 Tool — `generate_paraphrases`

Field	Type	Notes
`query`	string, required	the original question to vary
`n`	int, optional, default `4`	number of variants, 3–4 recommended
`include_marathi`	boolean, default `false`	see caution note below

Produces 3–4 variants spanning: **different phrasing** (same meaning, different words/word order), **formal register** ("What is the mandated open-space requirement…"), **informal register** ("how much playground area do I need for…"), and — only if `include_marathi=true` — a **Marathi translation**.

> **Caution:** bilingual support is confirmed at the intent-node level per the SRS, but it is not confirmed from Swagger alone whether `/chat` does full bilingual retrieval end-to-end. Verify with a manual Marathi query against staging before relying on `include_marathi=true` for a report finding.

Implementation: a single LLM call with a fixed instruction template ("produce N paraphrases of the following government-scheme question, preserving its exact meaning…"), temperature ~0.7 for lexical variety.

Variants are not sent to MahaSamvaad at this stage — that is `test_reformulation_stability`'s job.

Output

```
{
  "original_query": "…",
  "variants": [
    { "text": "…", "kind": "formal" },
    { "text": "…", "kind": "informal" },
    { "text": "…", "kind": "reorder" },
    { "text": "…", "kind": "marathi" }
  ]
}
```

7.2 Tool — `test_reformulation_stability`

Fires the original query plus every generated variant at `/chat` — via `query_and_log`, so each call also lands in the shared observability log — then diffs the returned `sources[]` across variants, comparing `gr_number` / `filepath` rather than the generated answer text (the phrasing of the answer is expected to vary; the cited document should not).

Field	Type	Notes
`query`	string, required	original query (also used to call `generate_paraphrases` internally if `variants` not supplied)
`variants`	array, optional	pre-generated variants; if omitted, `generate_paraphrases(query)` is called first
`web_search`	boolean, default `true`	forwarded to every underlying `/chat` call, held constant across variants so it is not a confound

Stability score = (count of variants whose top-ranked `source.gr_number` matches the majority `gr_number`) / (total variants fired). "3 of 4 phrasings cited the same top GR" → `stability_score = 0.75`, with the disagreeing variant returned as the odd one out.

Output

```
{
  "original_query": "…",
  "stability_score": 0.75,
  "majority_gr_number": "GR-2019-UD-…",
  "results": [
    { "kind": "original", "text": "…", "top_gr_number": "GR-2019-UD-…", "message_id": "…"
},
    { "kind": "formal",   "text": "…", "top_gr_number": "GR-2019-UD-…", "message_id": "…"
},
    { "kind": "informal", "text": "…", "top_gr_number": "GR-2019-UD-…", "message_id": "…"
},
    { "kind": "reorder",  "text": "…", "top_gr_number": "GR-2015-…",    "message_id":
"…", "odd_one_out": true }
  ]
}
```

A run summary is written to `reformulation_runs`, with one `reformulation_variants` row per fired variant, each carrying its own `message_id` back into `query_log` — so a reviewer can open the odd-one-out's full logged response for inspection.

Variants are fired concurrently with a bounded semaphore (max 3 in flight) rather than sequentially, to keep total run time reasonable without hammering staging (§9).

---

8. Cross-Server Interaction Model

MCP servers cannot call each other's tools directly — there is no server-to-server RPC in the protocol; only a client can invoke a tool on a server. Servers 2 and 3 both need to trigger `/chat` calls that get logged. Two implementation options — pick one explicitly rather than leaving it ambiguous:

Option A — shared library, three thin servers (recommended for v1). Factor the actual logging logic into `mcp_common/store.py` + `chat_client.py` (call the API, write the row). Server 1 exposes it as the `query_and_log` MCP tool; Servers 2 and 3 import the same functions directly in-process to fire their own `/chat` calls, so every server still writes to the one shared `store.db`. *Pro:* no runtime dependency on Server 1 being up; simplest to build and demo standalone. *Con:* the exposed `query_and_log` tool and Servers 2/3's internal calls are two entry points into the same function — keep them thin so they can't drift apart.

Option B — Server 3 as an MCP client of Server 1. Server 3 embeds a small MCP client (e.g. the Python `mcp` SDK's `ClientSession`) and connects to Server 1 as a subprocess/HTTP peer, calling its `query_and_log` tool over the protocol rather than importing code. *Pro:* exercises real MCP-to-MCP composition, closer to "any MCP-compatible agent" in the spirit of §5.4. *Con:* more moving parts, an extra process to keep alive, harder to debug on an intern timeline — treat as a Phase 5 hardening step (§13), not the v1 default.

---

9. Non-Functional Requirements

Concern	Requirement
Latency budget	`query_and_log` adds < 50ms overhead over the raw `/chat` call (logging happens after the response is captured, never blocking the return).
Concurrency	SQLite in WAL mode supports concurrent readers + one writer; wrap writes in a short retry-on-locked loop (`busy_timeout=5000ms`) since Servers 1–3 may write around the same time.
Timeouts	60s per `/chat` call (§4.2); `test_reformulation_stability` bounds concurrent variant calls to 3 in flight rather than firing all sequentially.
Idempotency	`message_id` is client-generated up front, so a logged row always exists even if the HTTP call itself fails — never silently drop a call.
Observability of the tools themselves	structured logging (`LOG_LEVEL`) to **stderr** per MCP convention — stdout is reserved for the protocol channel on stdio transport.
Data retention	`store.db` is an eval artifact, not user-facing storage — no automatic purge required for staging use, but do not point this suite at production traffic without a retention/PII review.

---

10. Error Handling & Edge Cases

- **Staging endpoint unreachable / 5xx** → `query_and_log` returns `ok:false` with the row still logged (`http_status` , `error_text` populated); dependent tools (`score_grounding` , `test_reformulation_stability`) must surface "upstream call failed for variant X" rather than crash the whole run.
- **`web_search=false` (corpus-only) responses** will have an empty `web_sources[]` by design — `get_dashboard_stats` and `grounding_trend` must not treat that as an anomaly.
- **Empty `sources[]` on a response** (e.g. off-topic/general intent per the SRS's intent taxonomy) → `score_grounding` short-circuits to `coverage_pct: null` (§6.1, step 6) rather than dividing by zero.
- **`message_id` not found** when calling `score_grounding` → return a clear `not_found` error rather than a generic exception; this is the most likely caller mistake (wrong id, or querying before the write completed).
- **Multilingual response text** feeding the sentence splitter (§6.1) — validate against at least one real Marathi-answer sample from staging before trusting `coverage_pct` on non-English responses.

---

11. Security Considerations

- The staging endpoint (`20.40.56.211`) as documented exposes no visible auth in Swagger — treat it as internal-network-only, non-production. Do not expose any of these three MCP servers on a public interface; bind to localhost or the internal staging network only.
- `user_id` / `chatroom_id` / message content may contain real user queries once this is pointed at anything beyond synthetic test traffic — `store.db` should be treated as containing potentially sensitive data and excluded from any public report or repo.
- No secrets are required for the base `/chat` call today; if the endpoint later requires an API key, load it from environment only (never hardcoded), consistent with `MAHASAMVAAD_BASE_URL` in §4.3.
- `generate_paraphrases` calls an external LLM — route through whatever provider key is already used elsewhere in the internship's tooling; do not embed a fresh key in this repo.

---

12. Testing & Validation Plan

Level	What to test
Unit	`chat_client.py` against a mocked `httpx` transport (timeout, 5xx, malformed JSON); sentence splitter against mixed Devanagari/Latin punctuation samples; stability-score arithmetic against hand-built source lists.
Integration (staging)	`GET /health` before any suite run; one real `query_and_log` call against a known GR question with a hand-verified expected department/GR number.
Server 1	`get_dashboard_stats` percentiles against a seeded `query_log` fixture with known latencies; `query_log://recent` pagination and filter params.
Server 2	`score_grounding` on a hand-labelled pair of (fully grounded, partially fabricated) sample responses, to sanity-check the default `0.55` threshold before trusting it.
Server 3	`test_reformulation_stability` against a GR question already known (from manual testing) to be sensitive to phrasing, to confirm odd-one-out detection actually fires.
End-to-end demo set	10–15 curated GR/Acts/Scheme questions spanning at least two departments, run through all three servers, forming the evidence base for the intern report.

---

13. Implementation Plan & Milestones

Phase	Deliverable
1	`store.db` schema + `mcp_common` (chat client, config); Server 1 skeleton with `query_and_log` wired end-to-end against staging.
2	`get_dashboard_stats` + `query_log://recent`; manually verify against a seeded log of ~30 real calls.
3	Server 2 (`score_grounding`, `grounding_trend`) with `difflib_v1`; hand-calibrate the `0.55` threshold on ~10 labelled samples.
4	Server 3 (`generate_paraphrases`, `test_reformulation_stability`); run the 10–15 question demo set end-to-end.
5	Hardening: concurrent-write retry, Option-B MCP-to-MCP composition (stretch), `embedding_v2` grounding (stretch), write-up of findings for the intern report.